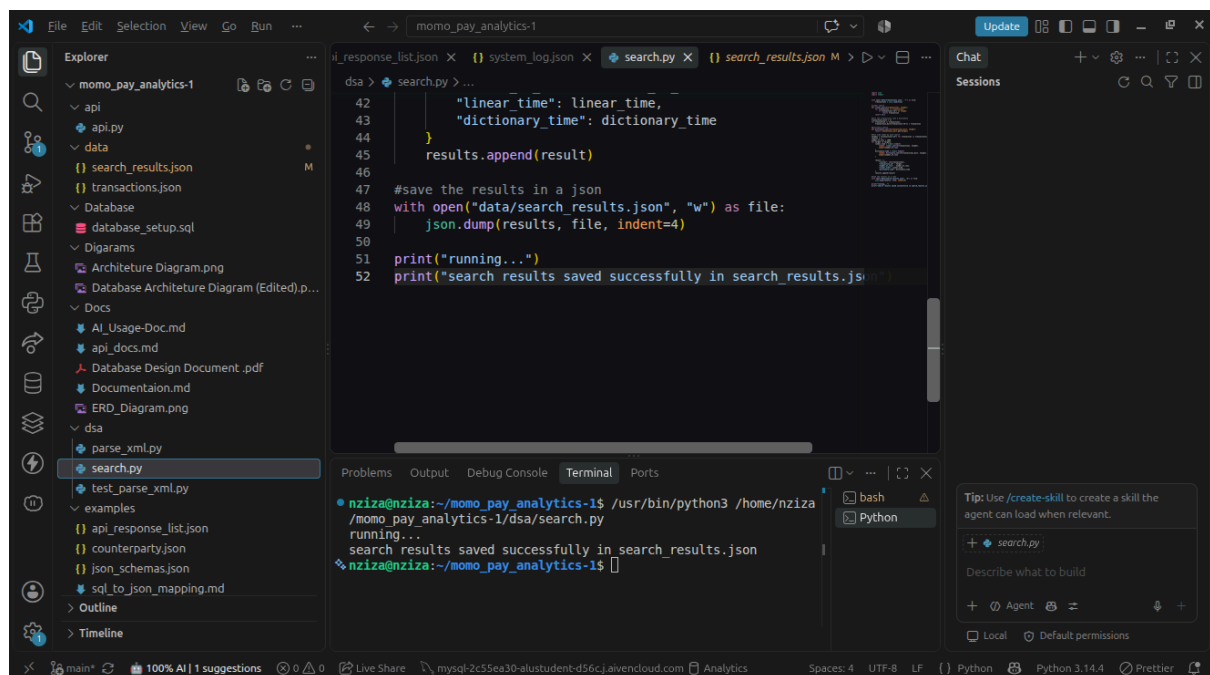


DSA Search Comparison Reflection

Linear Search vs Dictionary Lookup

I tested linear search and dictionary lookup using 1,683 transaction records. Each search method was tested 1,000 times for every transaction ID. The results showed that dictionary lookup was much faster overall than linear search.



```
File Edit Selection View Go Run ... momo_pay_analytics-1
Explorer
  momo_pay_analytics-1
    api
    api.py
    data
    search_results.json
    transactions.json
    Database
    database_setup.sql
    Digrams
    Architecture Diagram.png
    Database Architecture Diagram (Edited).p...
    Docs
    AI_Usage-Doc.md
    api_docs.md
    Database Design Document .pdf
    Documentaion.md
    ERD_Diagram.png
    dsa
    parse_xml.py
    search.py
    test_parse_xml.py
    examples
    api_response_list.json
    counterparty.json
    json_schemas.json
    sql_to_json_mapping.md
    Outline
    Timeline
  Problems Output Debug Console Terminal Ports
  Chat
  Sessions

dsa > search.py > ...
42     "linear_time": linear_time,
43     "dictionary_time": dictionary_time
44 }
45 results.append(result)
46
47 #save the results in a json
48 with open("data/search_results.json", "w") as file:
49     json.dump(results, file, indent=4)
50
51 print("running...")
52 print("search results saved successfully in search_results.js...")

Terminal
niziz@niziz:~/momo_pay_analytics-1$ /usr/bin/python3 /home/niziz/
momo_pay_analytics-1/dsa/search.py
running...
search results saved successfully in search_results.json
niziz@niziz:~/momo_pay_analytics-1$
```

The average time for linear search was approximately **0.0520 seconds per 1,000 runs**, while the average time for dictionary lookup was approximately **0.000177 seconds per 1,000 runs**. This shows a large difference in performance between the two methods.

```
data > {} search_results.json > {} 1675
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
{
  "records": 1683,
  "transaction_id": 1,
  "number_of_runs": 1000,
  "linear_time": 0.0029690459996345453,
  "dictionary_time": 0.0003079739981330931
},
{
  "records": 1683,
  "transaction_id": 2,
  "number_of_runs": 1000,
  "linear_time": 0.0007709940000495408,
  "dictionary_time": 0.0005390459991758689
},
{
  "records": 1683,
  "transaction_id": 3,
  "number_of_runs": 1000,
  "linear_time": 0.0013547730013669934,
  "dictionary_time": 0.0002978160009661224
},
{
  "records": 1683,
  "transaction_id": 4,
  "number_of_runs": 1000,
  "linear_time": 0.0010303629969712347,
  "dictionary_time": 0.0003321579970361199
},
{
  "records": 1683,
  "transaction_id": 5,
  "number_of_runs": 1000,
  "linear_time": 0.0010303629969712347,
  "dictionary_time": 0.0003321579970361199
}
```

Tip: Use /create-skill to create a skill the agent can load when relevant.

+ {} search_results.json

Describe what to build

+ Agent

Local Default permissions

Linear search checks the transactions one by one until it finds the required ID. Therefore, if the ID is near the end of the list, the search may need to check many records before finding it. This makes linear search $O(n)$, meaning the search time can increase as the number of records increases.

Dictionary lookup was much faster because the transaction IDs were used as dictionary keys. Python can directly access a value using its key, so it does not need to check every transaction one by one. Dictionary lookup has an average time complexity of $O(1)$, meaning the lookup time generally remains almost constant even when the number of records increases.

The results also show that dictionary lookup was relatively stable, while the linear search times varied more and generally became larger for transaction IDs that were further down the list. There were a few individual cases where linear search was slightly faster because of timing variation, but overall dictionary lookup was significantly faster.

```
data > {} search_results.json > {} 1675
7772 {
7773   "transaction_id": 1112,
7774   "number_of_runs": 1000,
7775   "linear_time": 0.07778190200042445,
7776   "dictionary_time": 0.00013685399972018786
7777 },
7778 {
7779   "records": 1683,
7780   "transaction_id": 1112,
7781   "number_of_runs": 1000,
7782   "linear_time": 0.07461152899850276,
7783   "dictionary_time": 0.00013822700202581473
7784 },
7785 {
7786   "records": 1683,
7787   "transaction_id": 1113,
7788   "number_of_runs": 1000,
7789   "linear_time": 0.08534856299957028,
7790   "dictionary_time": 0.0005292050009302329
7791 },
7792 {
7793   "records": 1683,
7794   "transaction_id": 1114,
7795   "number_of_runs": 1000,
7796   "linear_time": 0.17894818800050416,
7797   "dictionary_time": 0.00024218000180553645
7798 },
7799 {
7800   "records": 1683,
7801   "transaction_id": 1115,
7802   "number_of_runs": 1000,
7803   "linear_time": 0.08588621599847102,
7804   "dictionary_time": 0.0002605090012366418
7805 }
```

Further Improvement

Another data structure or algorithm that could improve search efficiency is **binary search**. Binary search works on sorted data and repeatedly divides the search area in half. Its time complexity is **$O(\log n)$** , which is faster than linear search for large datasets. Therefore, if the transaction IDs were stored in a sorted list, binary search could be used to make searching more efficient.

Conclusion

Based on my measured results, dictionary lookup was much faster than linear search for the transaction dataset. This is because dictionary lookup provides average **$O(1)$** access, while linear search can take **$O(n)$** time as the number of records increases. Binary search is another possible approach when the data is sorted because it can search in **$O(\log n)$** time.